

Low-Level Design (LLD)

1. Module-Level Design

The system is divided into functional, decoupled modules:

- **Document Processing Module:** Handles the ingestion of PDF files. Uses `PyPDFLoader` to extract raw text.
- **Chunking Module:** Implements `RecursiveCharacterTextSplitter` with a chunk size of 500 tokens and an overlap of 50 tokens to maintain semantic continuity.
- **Embedding Module:** Wraps the OpenAI `text-embedding-3-small` API to convert text chunks into 1536-dimensional float vectors.
- **Vector Storage Module:** Manages connection to the ChromaDB instance, handling insertion (upsert) and indexing of embeddings.
- **Retrieval Module:** Executes similarity searches (Cosine Similarity) against ChromaDB to fetch the top-k (e.g., k=3) relevant chunks based on the user's embedded query.
- **Query Processing Module:** Constructs the LLM prompt by combining the user's query and the retrieved context.
- **Graph Execution Module:** The LangGraph engine that compiles the nodes and edges, maintaining the state context across the query lifecycle.
- **HITL Module:** An asynchronous webhook/handler that pauses the graph execution and awaits external human input.

2. Core Data Structures

Defining the typed structures that move through the system:

A. Graph State Object (LangGraph)

This dictionary is passed and mutated between every node in the workflow.

```
class AgentState(TypedDict):  
    user_query: str  
    retrieved_context: List[str]  
    confidence_score: float  
    needs_human: bool  
    human_response: Optional[str]  
    final_answer: str
```

B. Chunking Format

```
{
  "chunk_id": "string (uuid)",
  "text": "string (extracted content)",
  "metadata": {
    "source": "filename.pdf",
    "page": "integer"
  }
}
```

3. Workflow Design (LangGraph)

The LangGraph implementation relies on specific nodes and edges:

• Nodes:

- `retrieve_node` : Queries ChromaDB and populates `retrieved_context` .
- `evaluate_node` : LLM evaluates intent and generates a draft answer and `confidence_score` .
- `hitl_node` : Suspends state, alerts agent.
- `generate_output_node` : Formats the final payload.

• Edges:

- Linear transition from `retrieve_node` to `evaluate_node` .
- **Conditional Edge** from `evaluate_node` to either `generate_output_node` or `hitl_node` based on the state variables.

4. Conditional Routing Logic

The conditional edge acts as the decision engine of the bot.

Answer Generation Criteria:

Proceed to `generate_output_node` IF:

1. `len(retrieved_context) > 0`
2. `confidence_score >= 0.85`

Escalation Criteria (Trigger HITL):

Route to `hitl_node` IF:

1. **Low Confidence:** `confidence_score < 0.85`.
2. **Missing Context:** ChromaDB returns an empty list or chunk similarity scores fall below the baseline threshold.
3. **Complex Query:** The LLM detects a multi-step intent requiring account access or sensitive policy interpretations.

5. HITL Design

The Human-in-the-Loop flow operates sequentially:

1. **Trigger:** LangGraph encounters the `hitl_node`.
2. **Interrupt:** The graph executes an interrupt (`NodeInterrupt`), pausing the thread and saving its exact state to a persistence layer (e.g., SQLite checkpoint).
3. **Alert:** A JSON payload containing the `user_query` and the reason for escalation is sent to the support dashboard.
4. **Integration:** A human agent provides an answer via the dashboard.
5. **Resume:** The dashboard API calls the LangGraph thread to resume, passing the human's text. The state updates `human_response` and routes to the `generate_output_node`.

6. API / Interface Design

How the UI communicates with the backend workflow:

Input Request:

```
{
  "session_id": "12345",
  "query": "How do I reset my account password?"
}
```

Output Response:

```
{
  "status": "success",
  "source": "bot", // or "human_agent"
  "answer": "To reset your password, navigate to..."
}
```

7. Error Handling

- **Missing Data:** If the user submits an empty query, the API returns a 400 Bad Request before hitting the LangGraph engine.
- **No Relevant Chunks Found:** The system gracefully escalates to the HITL module rather than hallucinating an answer.
- **LLM Failure:** If the OpenAI API times out, the system will implement an exponential backoff retry. If it fails 3 times, a fallback response ("System currently unavailable, please try again") is generated.